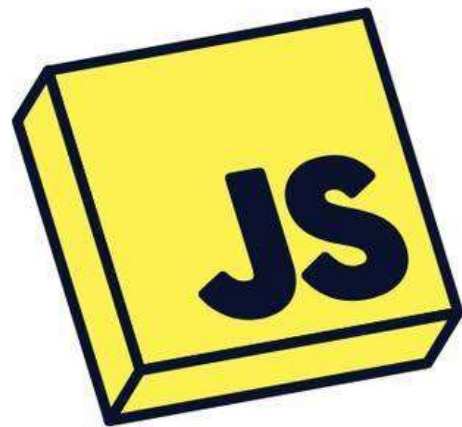


Top 20 JavaScript Concepts You Must Learn



Hoisting

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

Key Points:

- Works with var (hoisted but undefined) and function declarations (hoisted with definition).
- let and const are hoisted but remain in the Temporal Dead Zone (TDZ).

```
console.log(a); // undefined
var a = 5;

sayHello(); // "Hello!"
function sayHello() {
  console.log("Hello!");
}
```



The this Keyword

This refers to the object it belongs to, but its value depends on how a function is called.

Key Points:

- In a method: refers to the object.
- In a function: refers to undefined in strict mode, or window in non-strict mode.
- In arrow functions: inherits this from the surrounding scope

```
const obj = {  
  name: "John",  
  greet() {  
    console.log(this.name);  
  }  
};  
obj.greet(); // "John"
```



Promises

Promises handle asynchronous operations and represent a value that may be available now, later, or never.

Key Points:

- States: Pending, Fulfilled, Rejected.
- Methods: .then(), .catch(), .finally().

```
fetch('https://api.example.com/data')  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```



Async/Await

A syntactic sugar over promises, making asynchronous code look synchronous.

Key Points:

- Use await inside async functions.
- Easier error handling with try...catch.

```
async function getData() {  
  try {  
    const res = await fetch('https://api.example.com/data');  
    const data = await res.json();  
    console.log(data);  
  } catch (err) {  
    console.error(err);  
  }  
}
```



Event Loop

The mechanism that handles asynchronous callbacks in JavaScript.

Key Points:

- JS is single-threaded.
- Event loop checks the call stack and task queue.

```
console.log("Start");  
setTimeout(() => console.log("Async Task"), 0);  
console.log("End");
```



Call Stack & Execution Context

JavaScript uses the call stack to manage function execution, and each function runs inside its own execution context.

Key Points:

- Call Stack: LIFO (Last In, First Out).
- Execution Context has three phases: Creation, Execution, and Deletion.
- The Global Execution Context is created before any code runs.

```
function first() {  
  console.log("First");  
  second();  
}  
  
function second() {  
  console.log("Second");  
}  
  
first();  
// Stack flow: first() → second() → pop → pop
```



Prototypes & Inheritance

Every JavaScript object has a prototype from which it can inherit properties and methods.

Key Points:

- Prototype chain enables inheritance.
- `Object.create()` can create objects with a specific prototype.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.sayHi = function() {  
  console.log(`Hi, I'm ${this.name}`);  
};  
  
const user = new Person("John");  
user.sayHi(); // Hi, I'm John
```



Scope (Global, Function, Block)

Scope determines where variables can be accessed.

Key Points:

- Global Scope: accessible anywhere.
- Function Scope: variables declared inside functions are local.
- Block Scope: let and const are limited to {} blocks

```
if (true) {  
  let x = 10;  
}  
console.log(x); // ReferenceError
```



Lexical Scope

Lexical scope means inner functions can access variables from their parent functions.

Key Points:

- Scope is determined by where code is written, not where it's called.

```
function outer() {  
  let outerVar = "I'm outer";  
  function inner() {  
    console.log(outerVar);  
  }  
  inner();  
}  
outer();
```



Destructuring

Extract values from arrays or objects into variables in a cleaner way.

Key Points:

- Works with arrays, objects, and nested structures.



Spread & Rest Operators

... is used to expand or collect elements.

Key Points:

- Spread: expand arrays/objects.
- Rest: collect remaining values into a single array or object

```
const nums = [1, 2, 3];  
const newNums = [...nums, 4, 5];  
  
function sum(...args) {  
  return args.reduce((a, b) => a + b);  
}  
console.log(sum(1, 2, 3));
```



Variables (let, const, var)

JavaScript variables differ in scoping and mutability.

Key Points:

- var: function-scoped, hoisted.
- let: block-scoped, can be reassigned.
- const: block-scoped, cannot be reassigned

```
let x = 5;  
x = 10; // allowed  
const y = 15;  
y = 20; // TypeError
```



Closures

A closure is created when a function remembers its lexical scope even when executed outside its original scope.

Key Points:

- Useful for data privacy and state persistence

```
function makeCounter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```



Memory Management & Garbage Collection

JavaScript automatically allocates and frees memory when variables are no longer used.

Key Points:

- Uses reference counting and mark-and-sweep algorithm.
- Memory leaks happen if references aren't released

```
let obj = { name: "JS" };  
obj = null; // eligible for garbage collection
```



Debounce & Throttle

Techniques to control how often a function is executed.

Key Points:

- Debounce: runs after a pause in events.
- Throttle: runs at fixed intervals

```
function debounce(fn, delay) {  
  let timer;  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args), delay);  
  };  
}
```

